
GLA UNIVERSITY
ऋते ज्ञानान्न मुक्तिः
GLA
UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)
Accredited with A+ Grade by NAAC

QUEUE

Queue

- Queue is a Linear Non-Primitive Data Structure

- A **queue** is a first in, first out (**FIFO**) data structure.

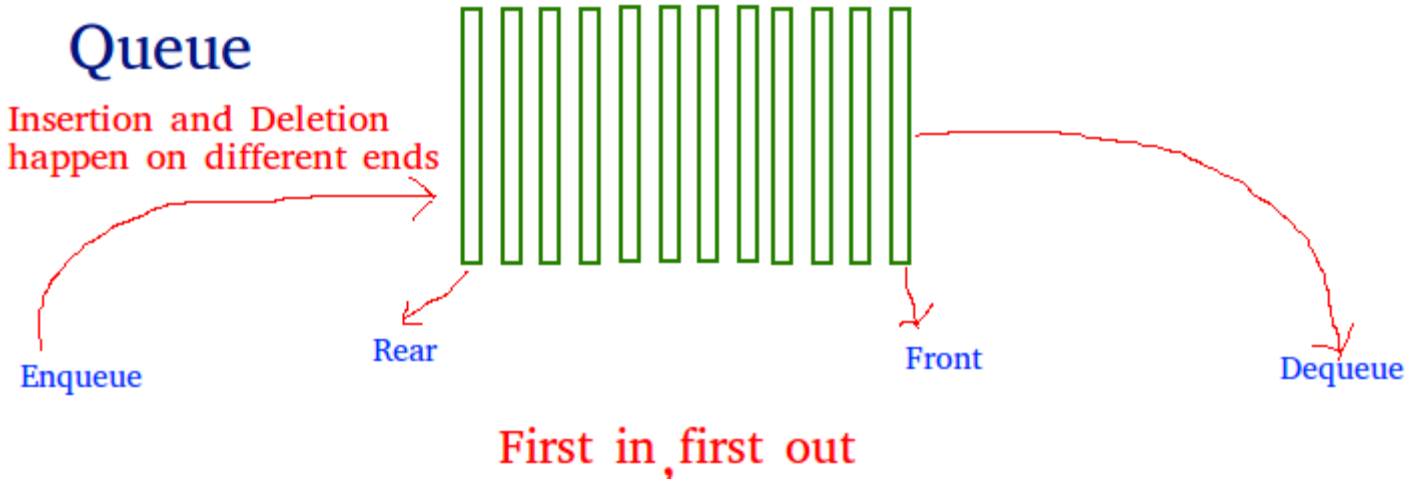
Cont…

- In a queue inserting will be done from one end **called rear end** and deletion will be done from other end called **front end**.

- Items are removed from a queue in the same order as they were inserted.

Cont…

- A good **example** of a queue is any queue of consumers for a resource where the consumer that came first is served first.



Cont…

- Queue can be implemented using **array** or **linked list**.

- Insert operation is called **enqueue**

- Delete operation is called **dequeue**

GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)
Accredited with A+ Grade by NAAC

Types of Queue

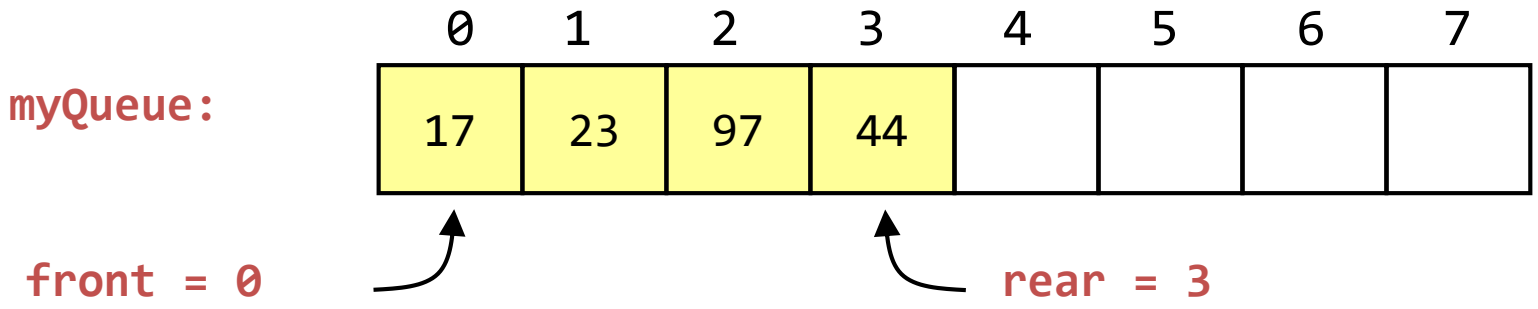
- Simple Queue (Linear Queue)
- Circular Queue
- Priority Queue
- Doubly Ended Queue (Deque)

Simple Queue (Linear Queue) Using Array

- Insertion occurs at the rear end of the queue and Deletions are performed at the front end of the queue.

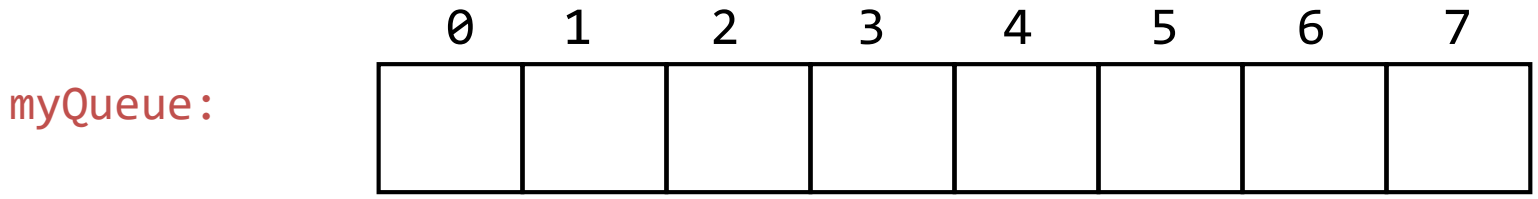
rear: it will contain the index of top most element.

front: it will contain the index of a element which will be deleted.



Linear Queue

- When Queue is empty
- If (front == rear == -1)
  - Queue is empty

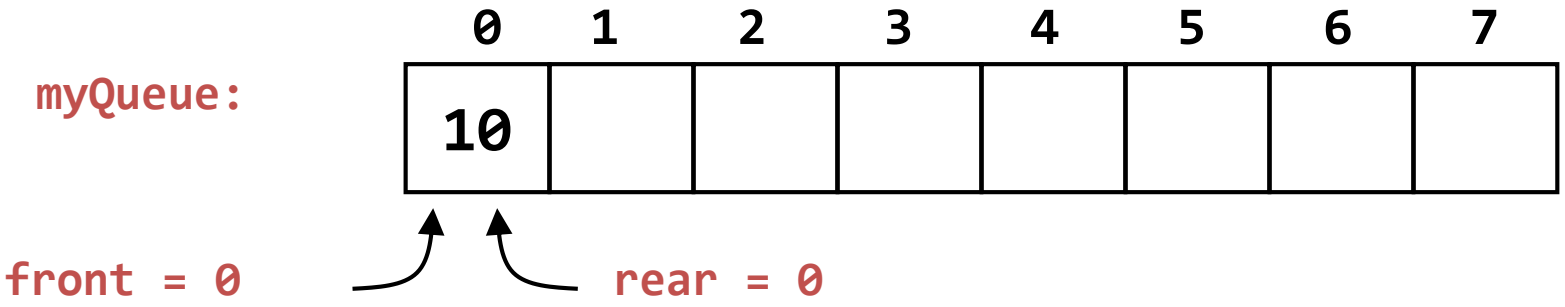



```
front = -1
rear = -1
```

Linear Queue

- **Let insert first element** 10 **in queue** myQueue

```
front = -1
rear = -1
```

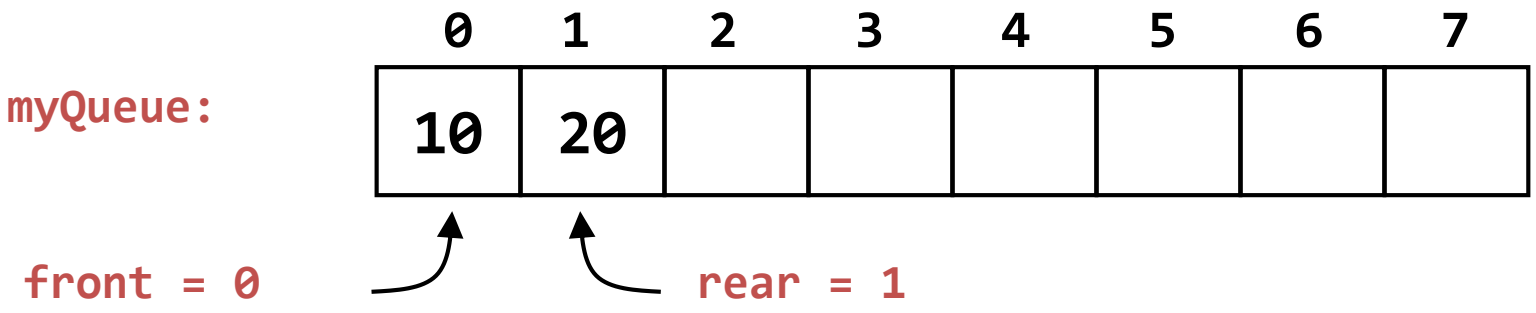



```
front = front +1    = -1+ 1 = 0
rear = rear +  1    = -1+1 = 0
myQueue[rear] = myQueue[0] = 10
```

Linear Queue

- **Let insert second element** 20 **in queue** myQueue

```
front = 0        front = 0
rear = 0         rear = rear + 1    = 0 + 1 = 1
```

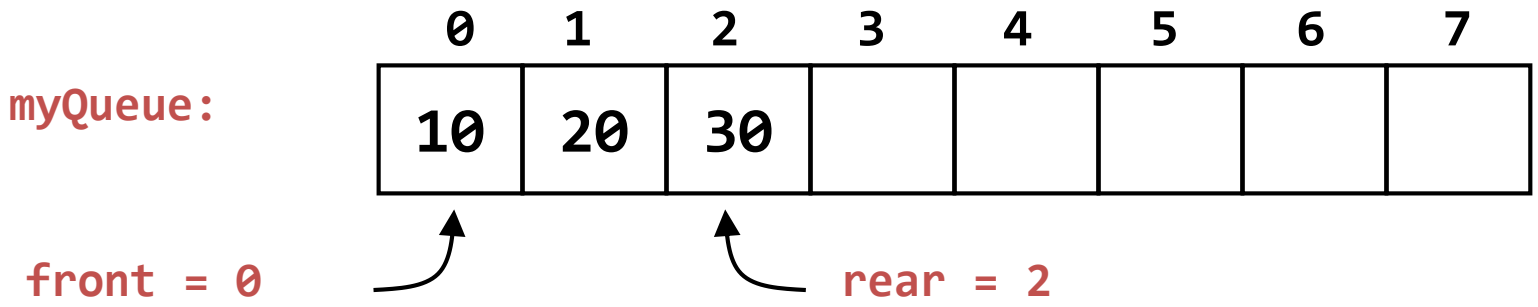



```
myQueue[rear]= myQueue[1] = 20
```

Linear Queue

- **Let insert third element** 30 **in queue** myQueue

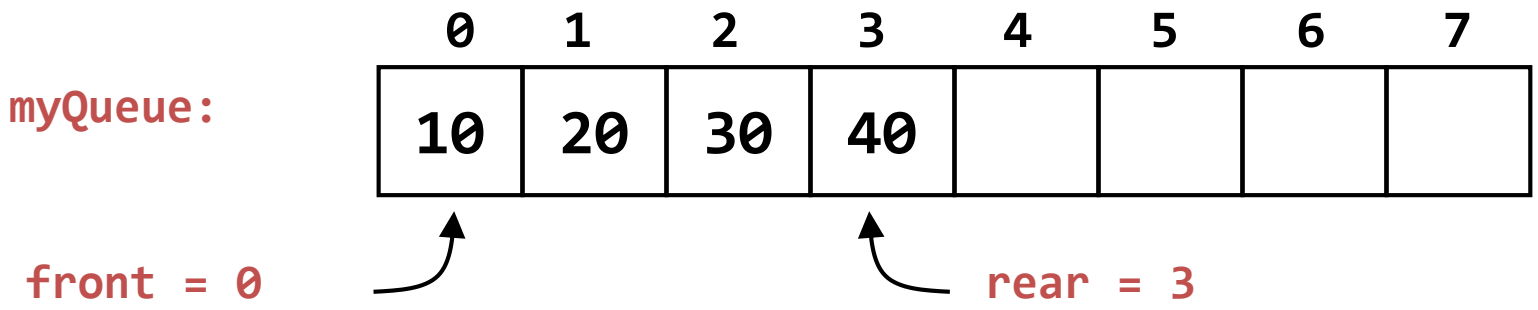
```
front = 0      front = 0
rear = 1        rear = rear +  1     = 1+1 = 2
```



Linear Queue

- **Let insert fourth element** 40 **in queue** myQueue

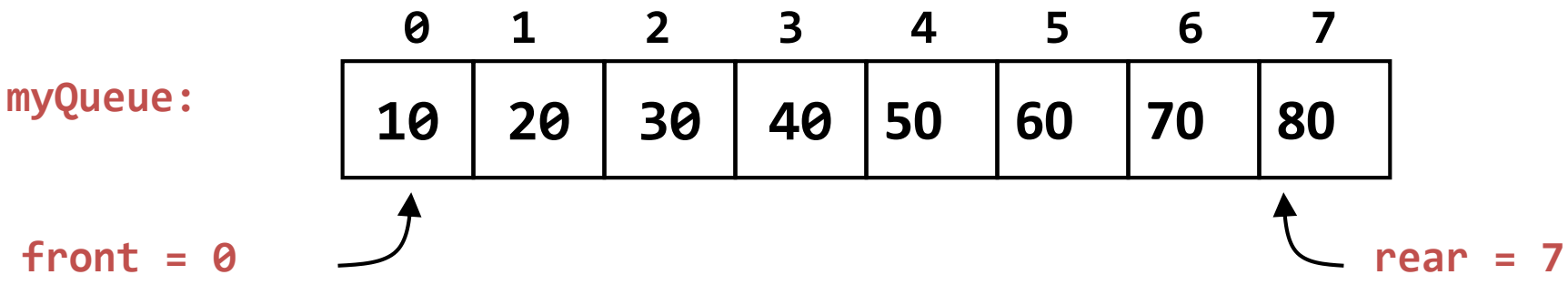
```
front = 0      front = 0
rear = 2        rear = rear +  1     = 2+1 =3
```



Linear Queue

- **Let insert last element** 80 **in queue** myQueue

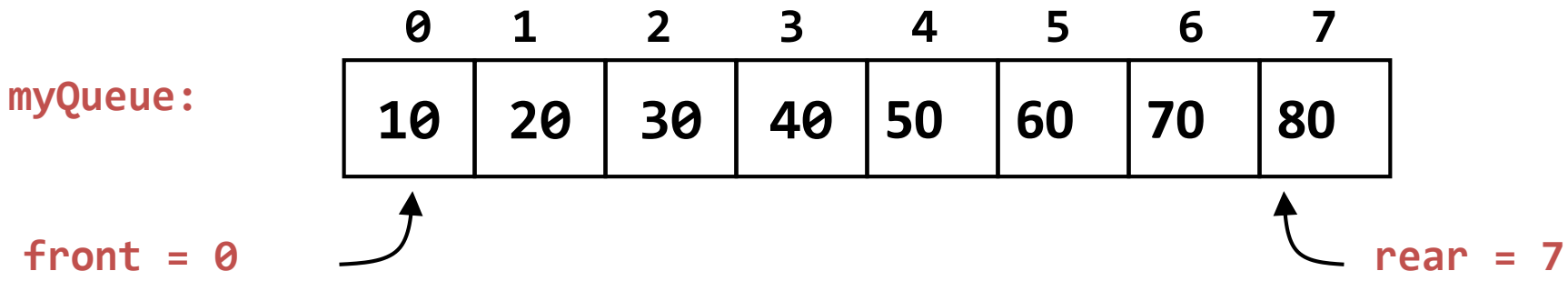
```
front = 0      front = 0
rear = 6        rear = rear +  1 = 6+1    = 7
```



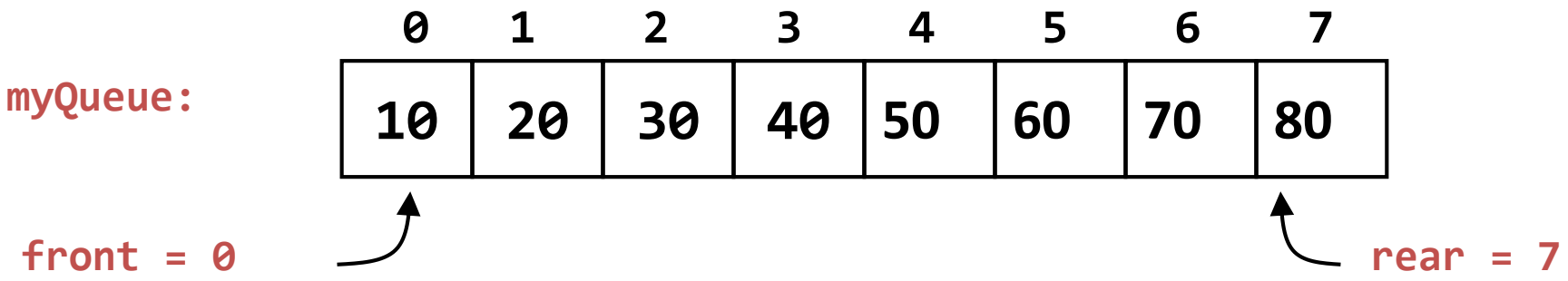
Linear Queue

- **Now the** myQueue is full.
- if (rear == Maxsize – 1)

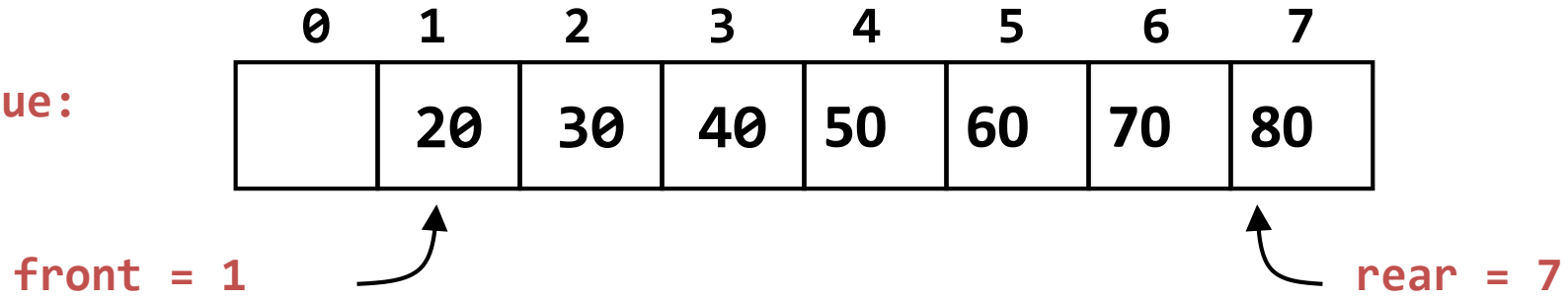
  Queue is full



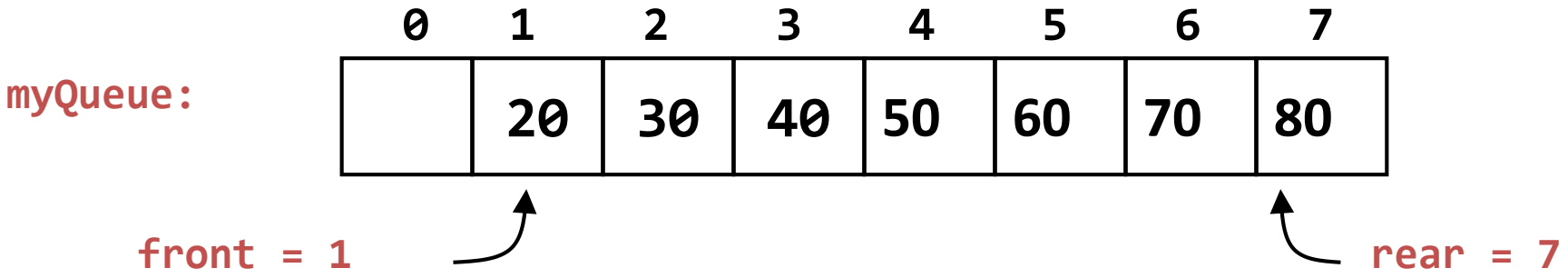
Delete one element from Linear Queue



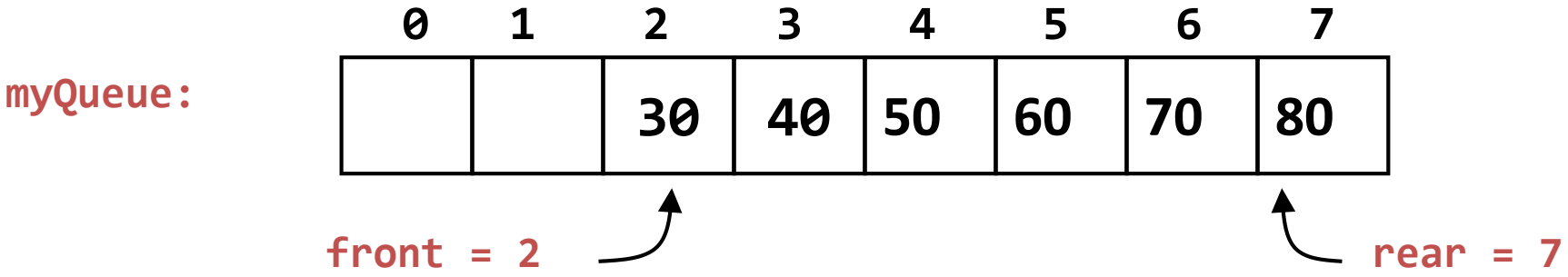
- **10 will be deleted first** as it has come first in the queue



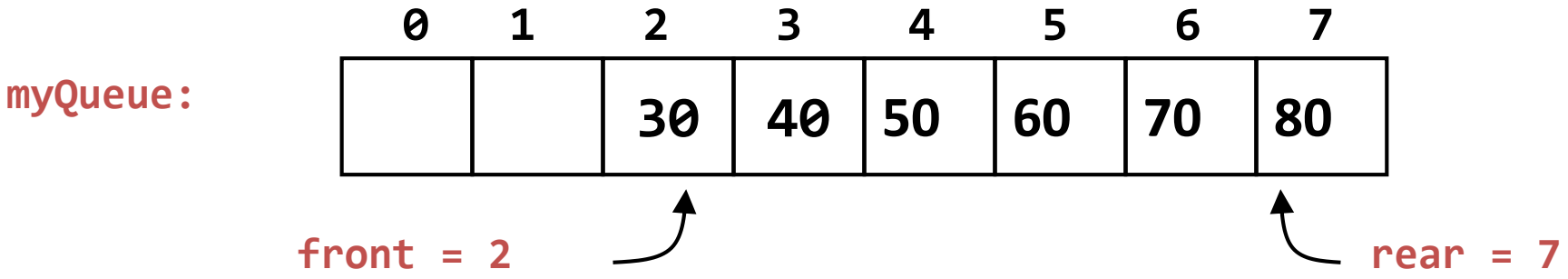
Delete one element from Linear Queue



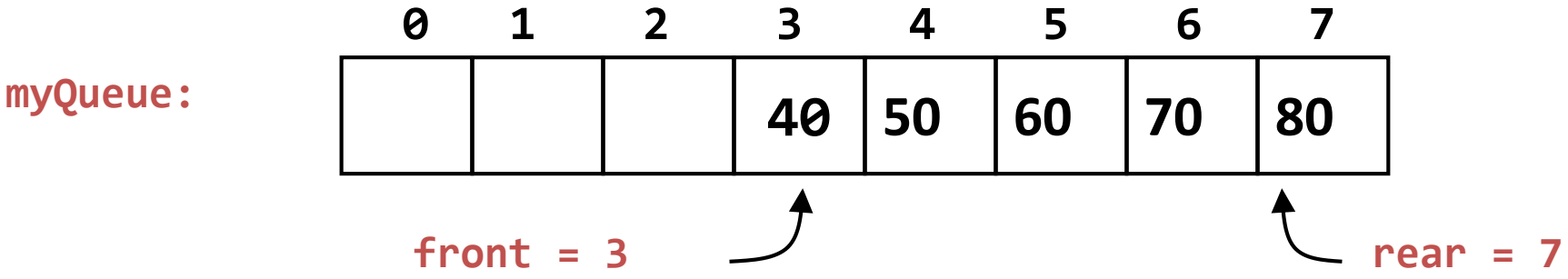
- 20 will be deleted next



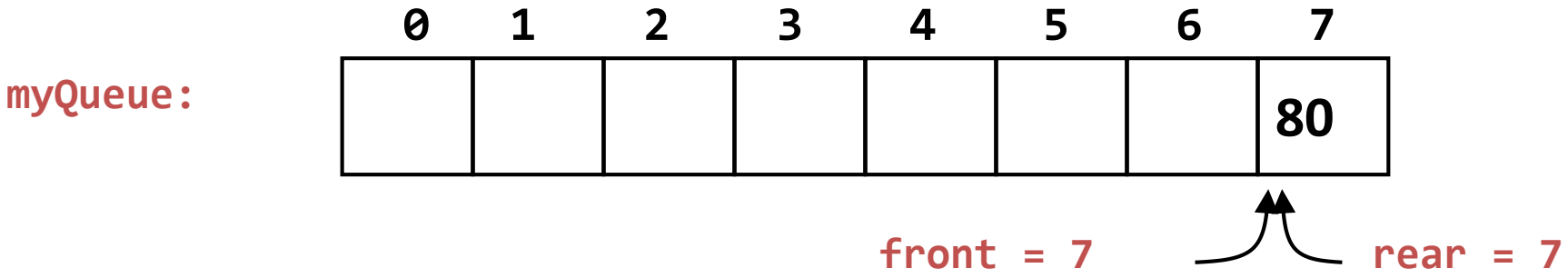
Delete one element from Linear Queue



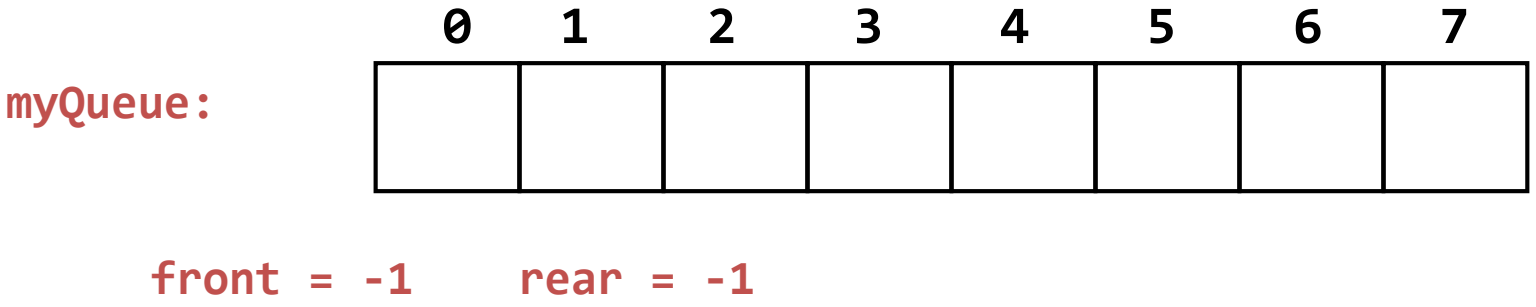
- **Now 30 will be deleted**



Delete one element from Linear Queue



- **Now 80 will be deleted**



Linear Queue using Array

Algorithm: insert operation

Input: **(Queue[MaxSize], rear, front, data)**

Here Queue is an array of maximum size Maxsize.

rear: it will contain the index of top most element.

front: it will contain the index of a element which will be deleted.

This algorithm insert the element data at rear end.

Step 1: Start

Step 2: if (rear == MaxSize-1)

```
{
   print("queue is full");
    exit(0);
}
```

Cont…

```
else if (rear==-1)
  {
     rear = 0;
     front = 0;
     }
else
   {
       rear= rear+1;
     }
     Step 3: Queue[rear] = data;
     Step 4: Stop
```

Linear Queue using Array

Algorithm: delete operation

Input: (**Queue[MaxSize], rear, front)**

Here Queue is an array of maximum size Maxsize.

rear: it will contain the index of top most element.

front: it will contain the index of a element which will be deleted.

This algorithm delete the element from front end.

Step 1: Start

Step 2: if (front==-1)

```
{
    printf("queue is empty");
}
```

Cont…

```
else if (rear == front )
   {
      rear = -1;
      front = -1;
    }
else
 {
   front = front+1;
 }
```

Step 3: Stop

Linear Queue using Array

Algorithm: Displaying all elements of queue

Input: (**Queue[MaxSize], rear, front)**

Here Queue is an array of maximum size Maxsize.

rear: it will contain the index of top most element.

front: it will contain the index of a element which will be deleted.

This algorithm displaying all elements of queue.

Step 1: Start

Step 2: if (front == -1)

{

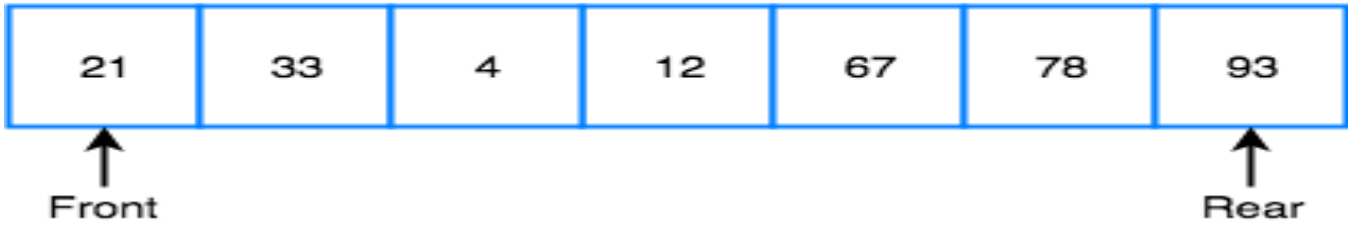
Cont…

```
        printf("queue is empty");
}
 else
  {
     for (i = front ; i<=rear; i++)
     print("%d",Queue[i]);
  }
```

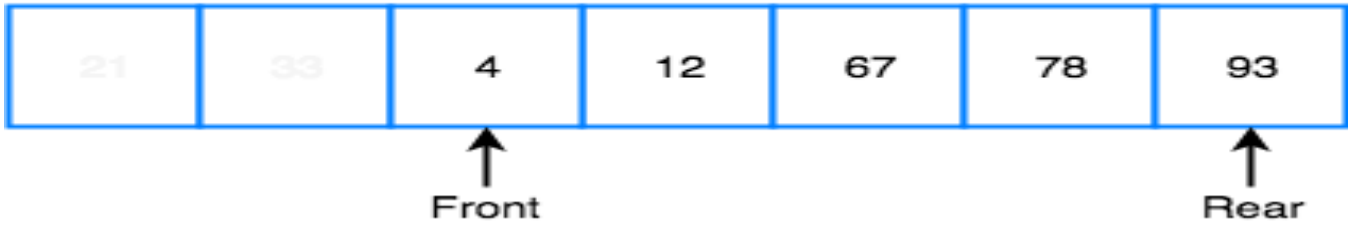
Step 3: Stop

Limitation of Linear Queue

Queue is Full



Queue is Full (Even after removing 2 elements)



Limitation of Linear Queue

•In a Linear queue, once the queue is completely full, it's not possible to insert more elements.

•Now, actually we have deleted 2 elements from the queue so, there should be space for another 2 elements in the queue, but as the rear is pointing at last position and Queue overflow condition
(Rear == MAXSIZE-1) is true so we can't insert the new element in the queue even if it has an empty space.

Cont…

- To overcome this problem there is another variation of queue called CIRCULAR QUEUE.

Question

Example:

If the elements “A”, “B”, “C” and “D” are placed in a queue and are deleted one at a time, in what order will they are removed?

Circular Queue

- **Circular queue** avoids the wastage of space in a Linear queue implementation using arrays.

- A **circular queue** is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle and the last position is connected back to the first position to make a circle.

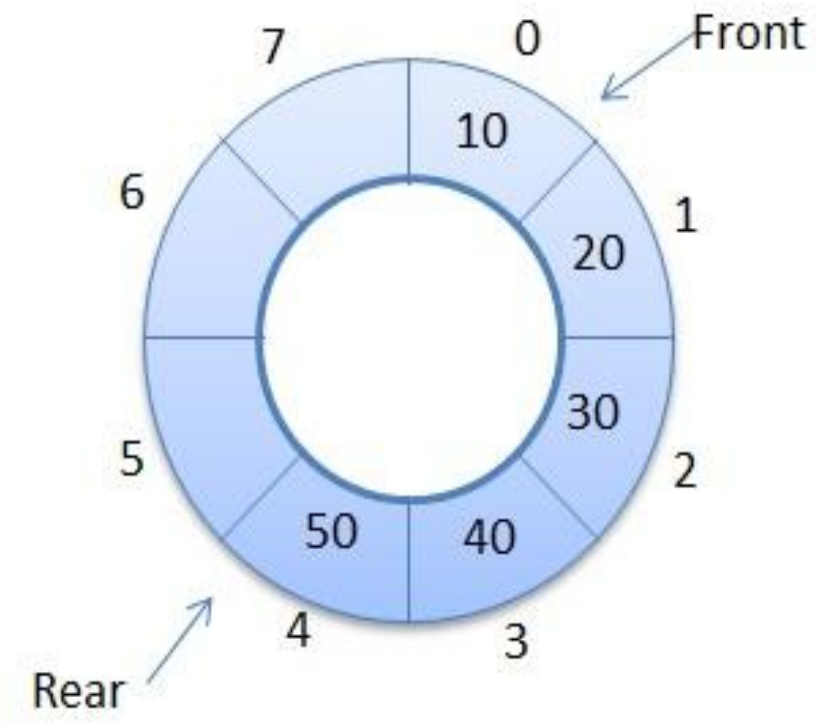
Cont…

- rear = (rear + 1) % SIZE
- front=(front+1) % SIZE

- if(front == -1)
  - Circular Queue is empty

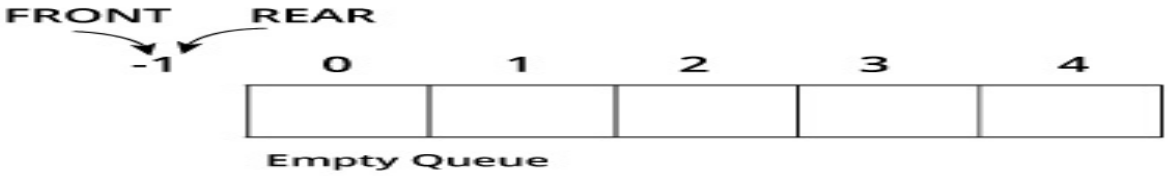
- If (front == (rear + 1) % SIZE)
  - Circular Queue is full

Circular Queue Examples

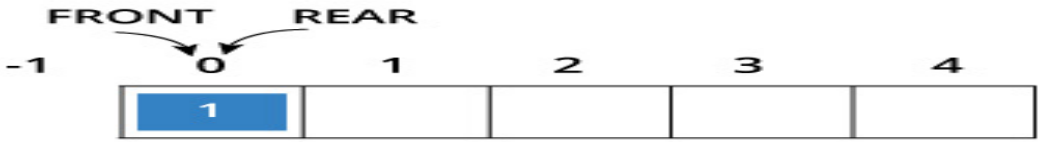


GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)
Accredited with A+ Grade by NAAC

Circular Queue Example

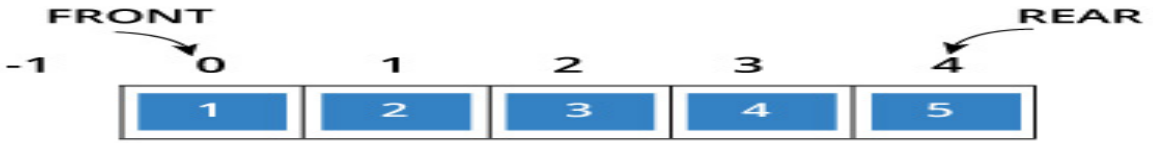


Empty Queue



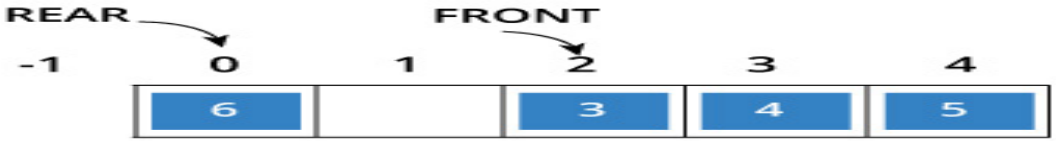
EnQueue first element

EnQueue



EnQueue

DeQueue



EnQueue

Queue Full

Algorithm to insert an element in circular queue using Array

Input: (Queue[MaxSize], rear, front, data)

Here Queue is an array of maximum size Maxsize.

rear: it will contain the index of top most element.

front: it will contain the index of a element which will be deleted.

This algorithm insert the element data at rear end.

Step 1: Start

Step 2: if (front == (rear + 1) % MaxSize)

```
{
   print("circular queue is full");
}
```

Cont…

```
else if (rear==-1)
  {
     rear = 0;
     front = 0;
     Queue[rear] = data;
     }

else
   {
       rear = (rear + 1) % MaxSize ;
      Queue[rear] = data;

     }
```

Step 3: Stop

Algorithm to delete an element in circular queue using Array

Input: (Queue[MaxSize], rear, front)

Here Queue is an array of maximum size Maxsize.

rear: it will contain the index of top most element.

front: it will contain the index of a element which will be deleted.

This algorithm delete the element from front end.

Step 1: Start

Step 2: if (front==-1)

```
{
    printf("circular queue is empty");
}
```

Cont…

```
else if (rear == front )
   {
      rear = -1;
      front = -1;
    }
else
 {
   front=(front+1) % MaxSize;
 }
```

Step 3: Stop

Question

Question: Consider the following queue of characters, where QUEUE is circular queue which is allocated six memory cells with FRONT = 2, REAR = 4 and QUEUE = _ , A, C, D, _ , _

- "_" indicates empty memory cell. Following operations are performed on the circular queue.

Cont…

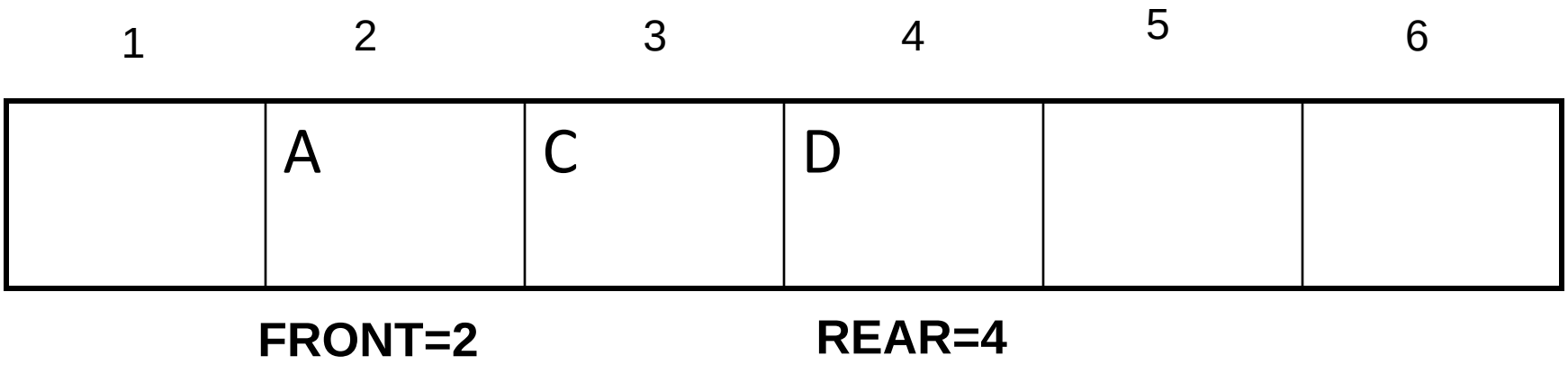
(i) F is added to the queue.
(ii) Two letters deleted.
(iii) K, L and M are added.
(iv) Two letters deleted.
(v) R is added.
(vi) Two letters deleted.
(vii) S is added.
(viii) Two letters deleted.
(ix) One letter deleted.
(x) One letter deleted

What will be character deleted from the circular queue in operation (ix) and (x)?

Solution

Initially

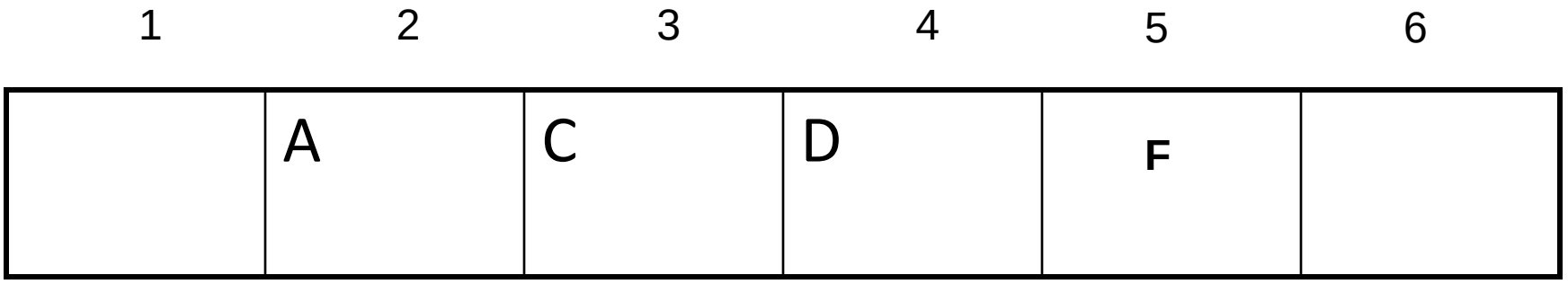
Q



Cont…

(I) F is added to the queue.

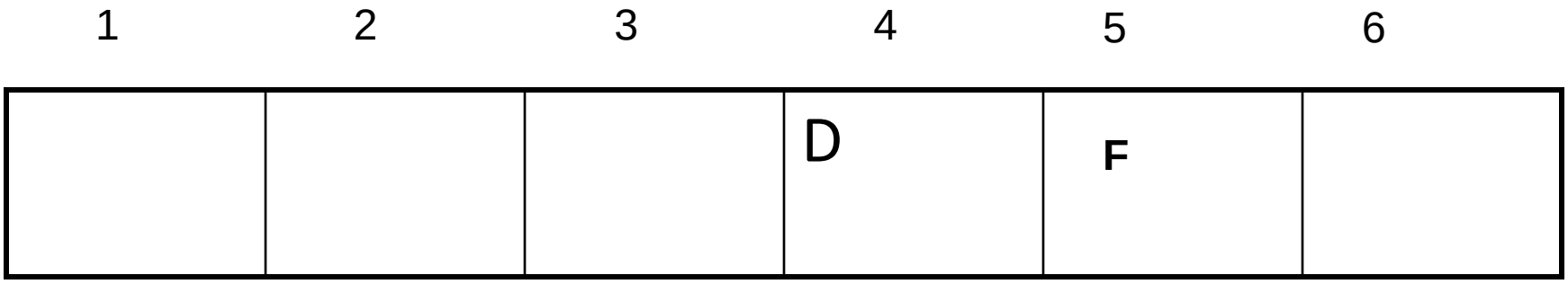
Q **REAR=4**



(II) Two letters deleted.

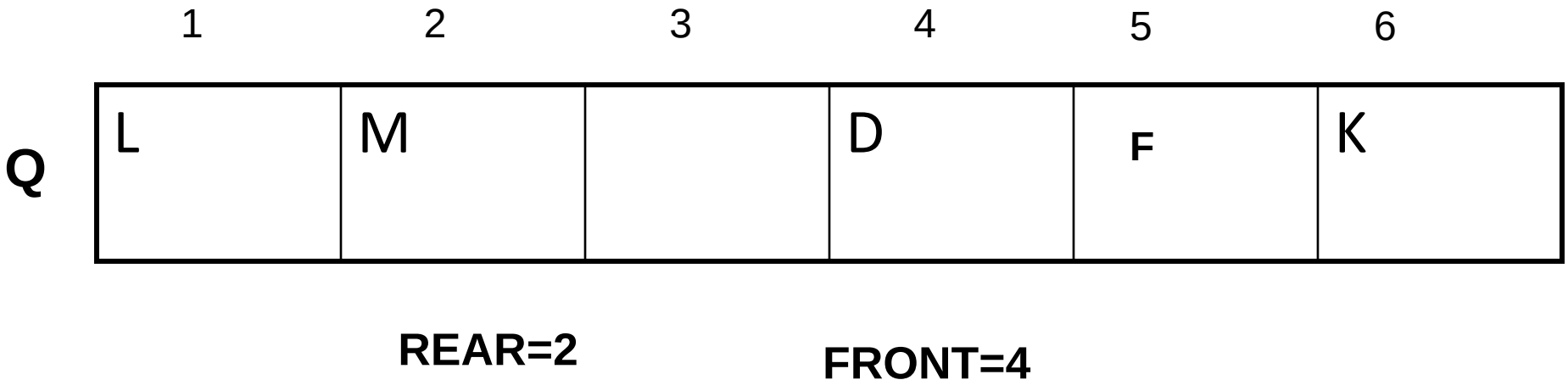
FRONT =2

Q



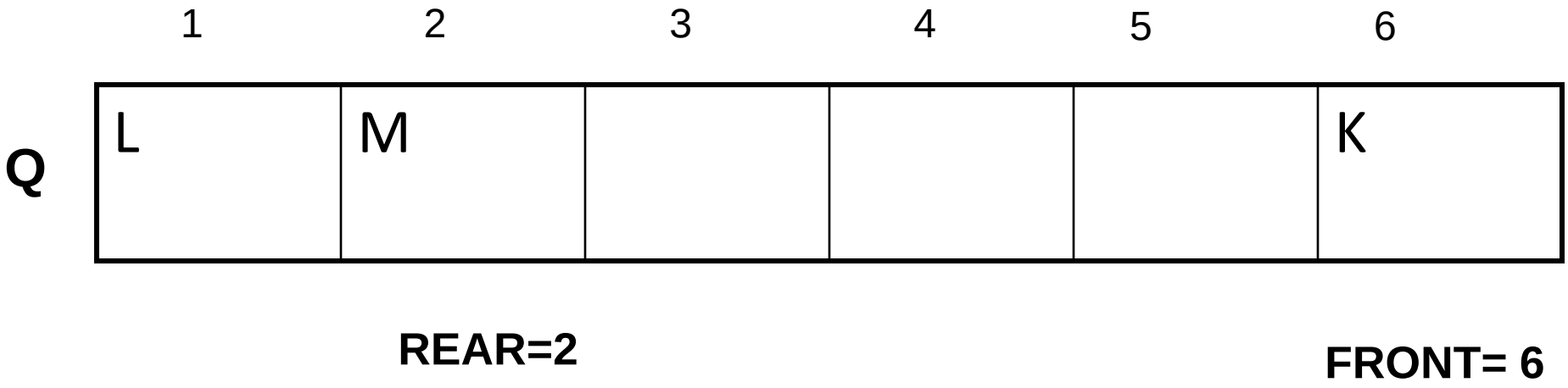
(III) K, L and M are added.

REAR =4



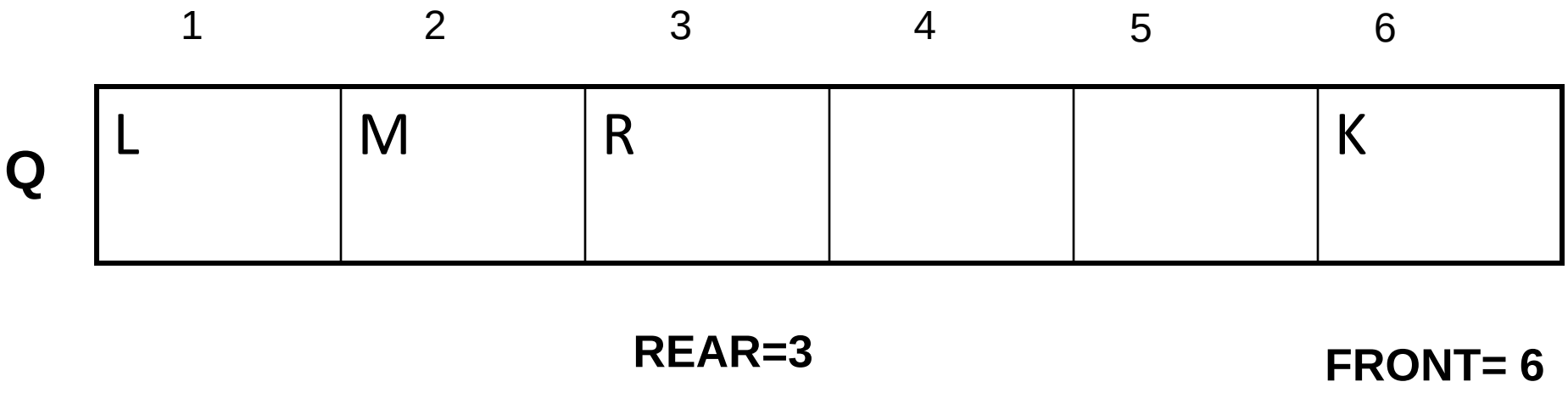
(IV) Two letters deleted.

FRONT =4

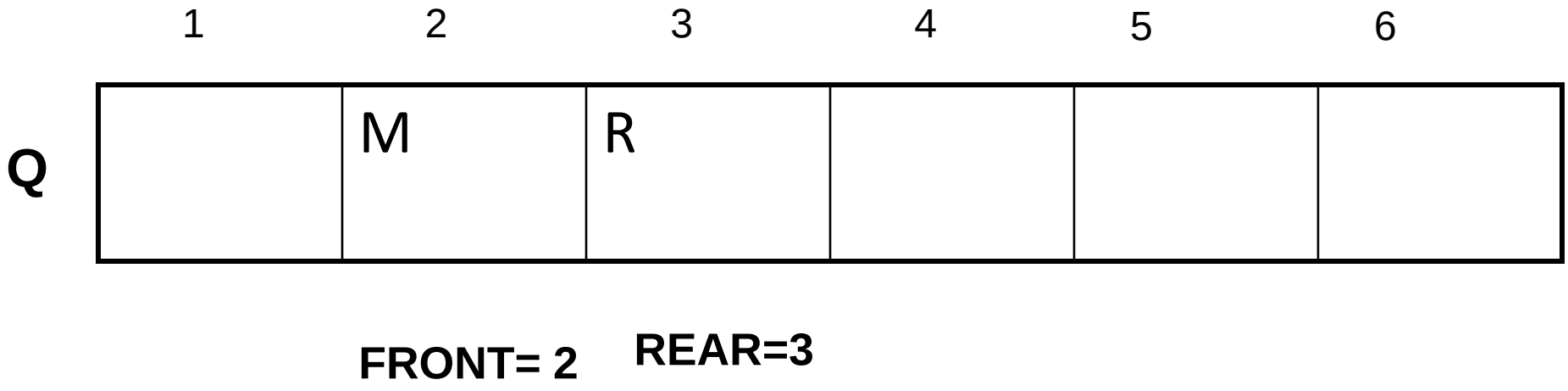


(V) R is added.

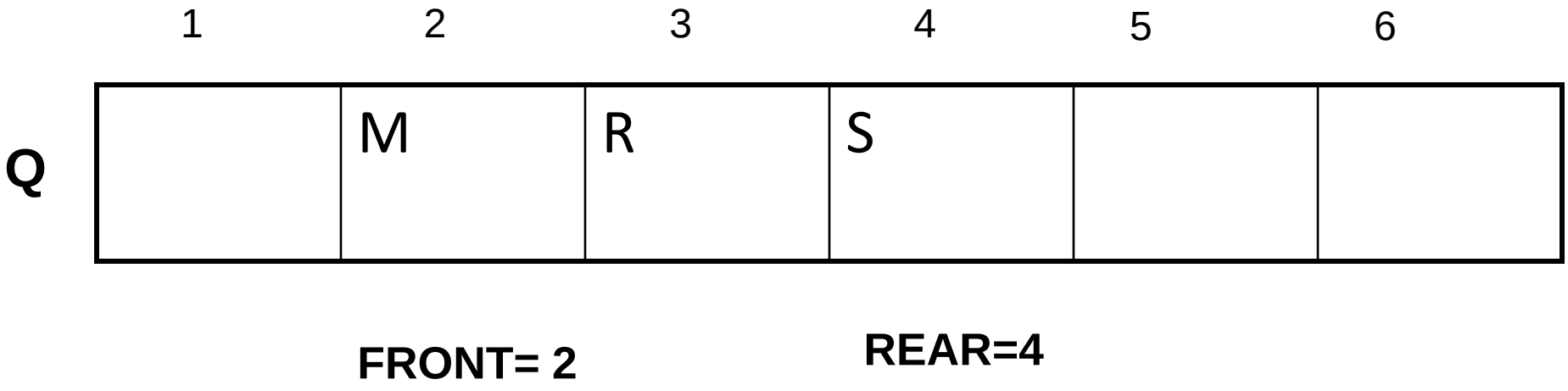
REAR =2



(VI) Two letters deleted.
FRONT =6

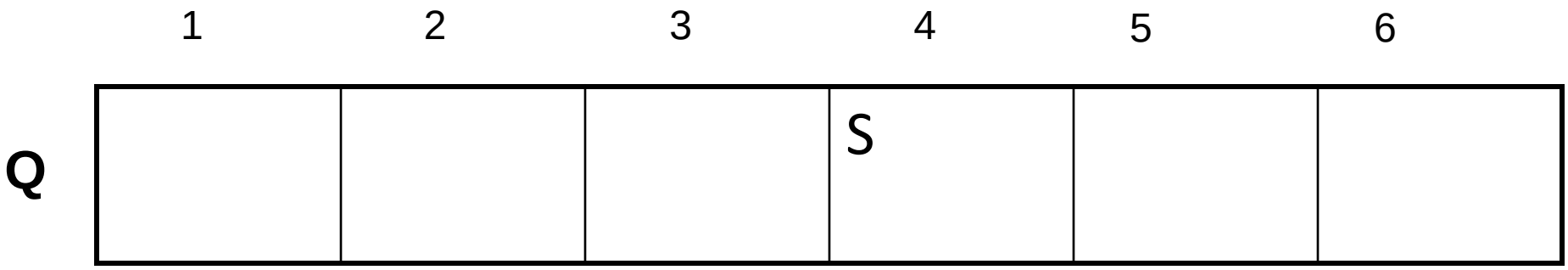


(VII) S is added.
REAR =3



(VIII) Two letters deleted.

FRONT = 2

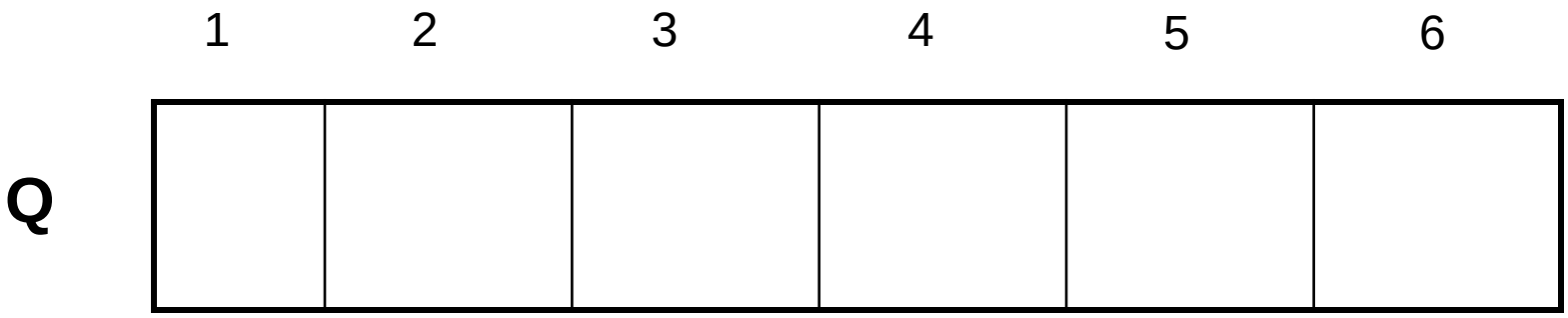


REAR=4

FRONT= 4

(IX) One letter deleted. And in this operation S will be deleted.

FRONT = 4

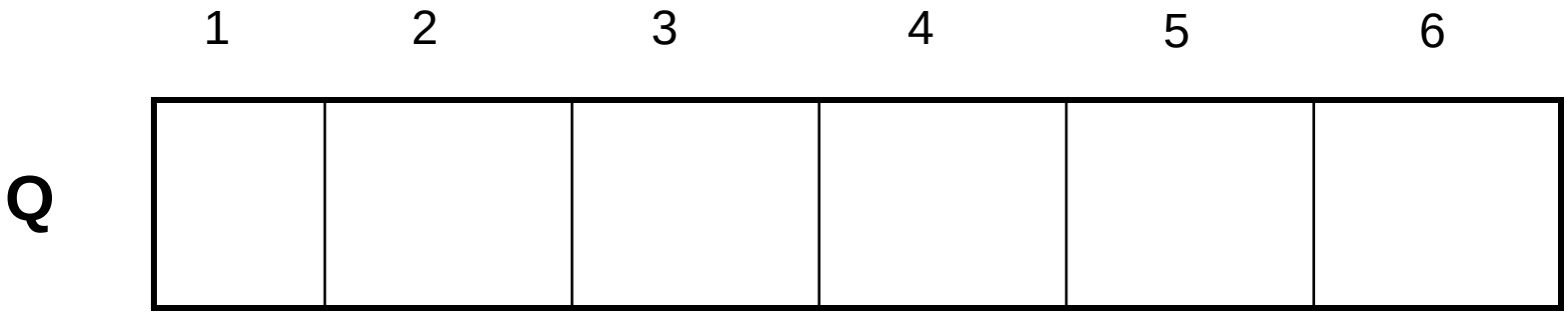


REAR= 0

FRONT= 0

(X) One letter deleted.

FRONT = 0
Circular Queue is empty so we can not delete one letter.



REAR= 0

FRONT= 0

Cont…

- **S will be character deleted from the circular queue in operation (ix).**

- **In (x) operation FRONT = 0 so Circular Queue is empty so There will be no character deleted from the circular queue in operation (x).**

Priority Queue

- Priority Queue is an extension of queue with following properties.

1. Every item has a priority associated with it.
2. An element with high priority is dequeued (delete) before an element with low priority.
3. If two elements have the same priority, they are served according to their order in the queue.

GLA UNIVERSITY MATHURA
Accredited with A+ Grade by NAAC

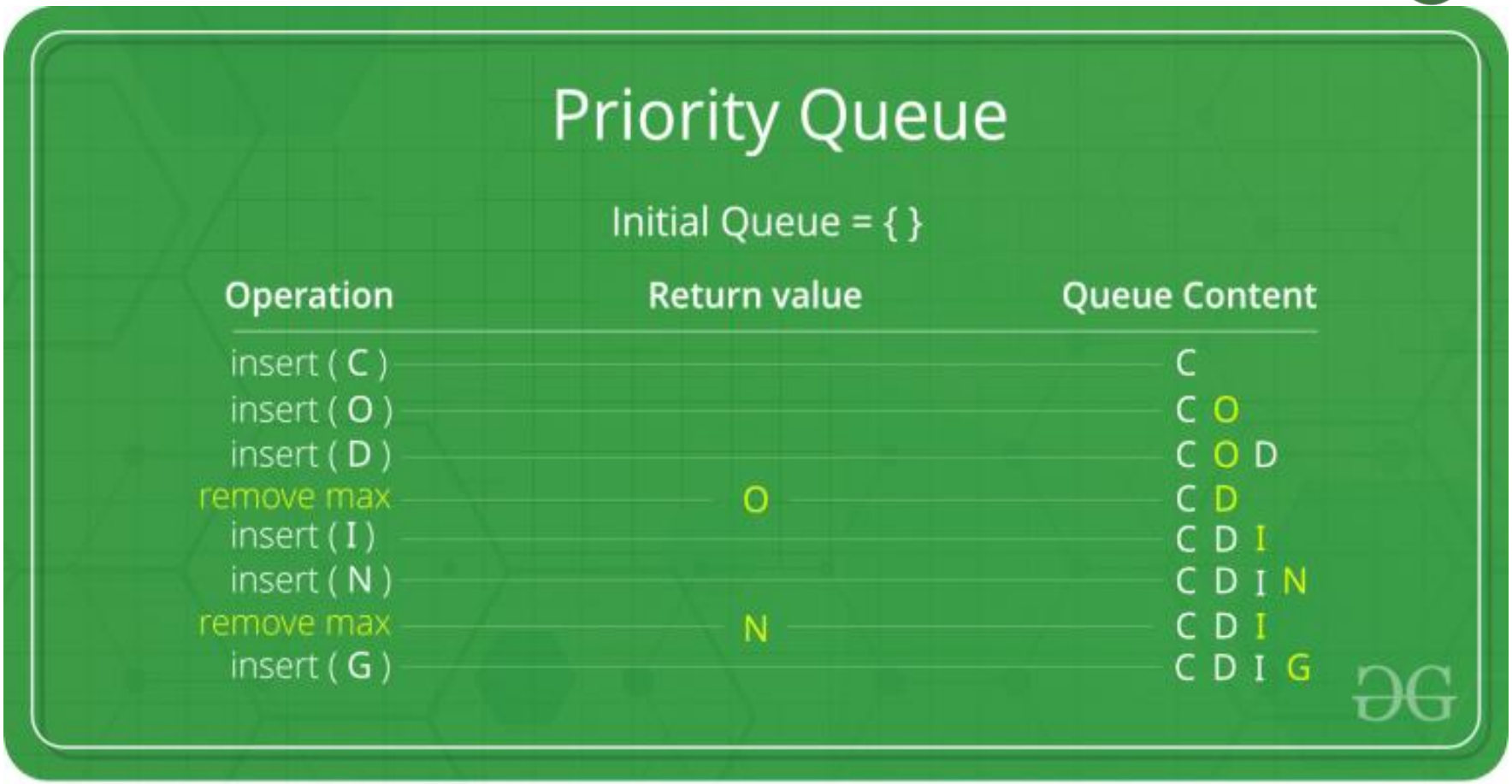


Figure : 1

In the Above Figure:1 priority queue, element with maximum ASCII value will have the highest priority.

Cont…

Types of Priority: There are two types of priority

1. **Implicit Priority:** The priority of element is decided by its value is known as implicit priority.

   Example: Assume that we insert the element in the order 8, 3, 2 & 5. Here we are not given the priority for the element.

Cont…

2. **Explicit Priority:** When priority number is assignment with each element is known as explicit priority.

Element	10	4	15	11
Priority	2	6	4	12

Types of Priority Queue

- There are two types of priority queues they are as follows...

1. **Max Priority Queue**
2. **Min Priority Queue**

1. **<u>Max Priority Queue</u>:**

- In a max priority queue, elements are inserted in the order in which they arrive in the queue.

Cont…

- the maximum value is always removed first from the queue.

For example: Assume that we insert in the order 8, 3, 2 & 5 and they are removed in the order 8, 5, 3, 2 .

Cont…

- The following are the operations performed in a Max priority queue...
- **isEmpty() - Check whether queue is Empty.**
- **insert() - Inserts a new value into the queue.**
- **findMax() - Find maximum value in the queue.**
- **remove() - Delete maximum value from the queue.**

GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)
Accredited with A+ Grade by NAAC

2. Min Priority Queue:

- In a min priority queue, elements are inserted in the order in which they arrive in the queue.

- the minimum value is always removed first from the queue.

Cont…

For example: Assume that we insert in the order 8, 3, 2 & 5 and they are removed in the order 2, 3, 5, 8.

Cont…

- The following operations are performed in Min Priority Queue...
- **isEmpty()** - Check whether queue is Empty.
- **insert()** - Inserts a new value into the queue.
- **findMin()** - Find minimum value in the queue.
- **remove()** - Delete minimum value from the queue.

Applications of Priority Queue:

1) CPU Scheduling.

2) Graph algorithms like Dijkstra's shortest path algorithm, Prim's Minimum Spanning Tree, etc

3) All queue applications where priority is involved.

GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)
Accredited with A+ Grade by NAAC

Example:

Define priority queue. Consider an empty circular queue Q[_ _ _ _ _] of size N=5. Perform the following operations:-

(a) Enqueue D,E and F

(b) Dequeue

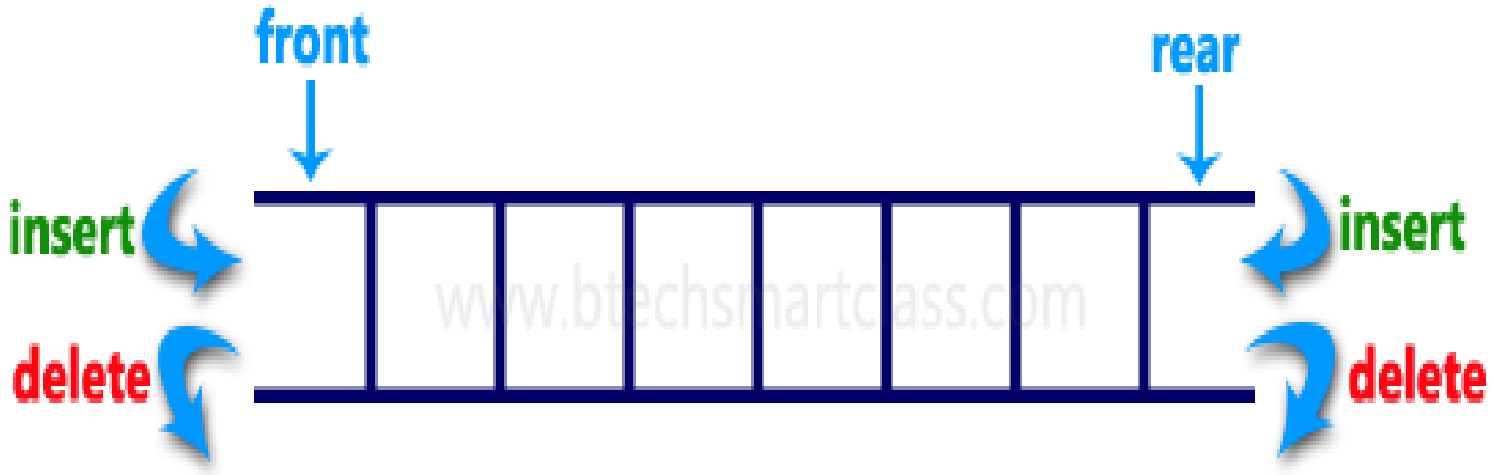
(c) Enqueue B and A

(d) Dequeue

(e) Enqueue H, C and D

Deque or Double Ended Queue

- Double Ended Queue is also a Queue data structure in which the insertion and deletion operations are performed at both the ends (**front** and **rear**).
- That means, we can insert at both front and rear end and can delete from both front and rear end.



Cont…

- There are four possible operations that can be performed on a deque.

1. Insert at rear end.
2. Delete at rear end.
3. Insert at front end.
4. Delete at front end.

Cont…

NOTE:

1. If you insert the element at rear end than firstly increase the rear by one i.e

   rear = rear +1 than insert element at rear end.

2. If you delete the element from rear end than decrease the rear by one i.e

   rear = rear – 1 .

Cont…

3. If you delete the element from front end than increase the front by one i.e

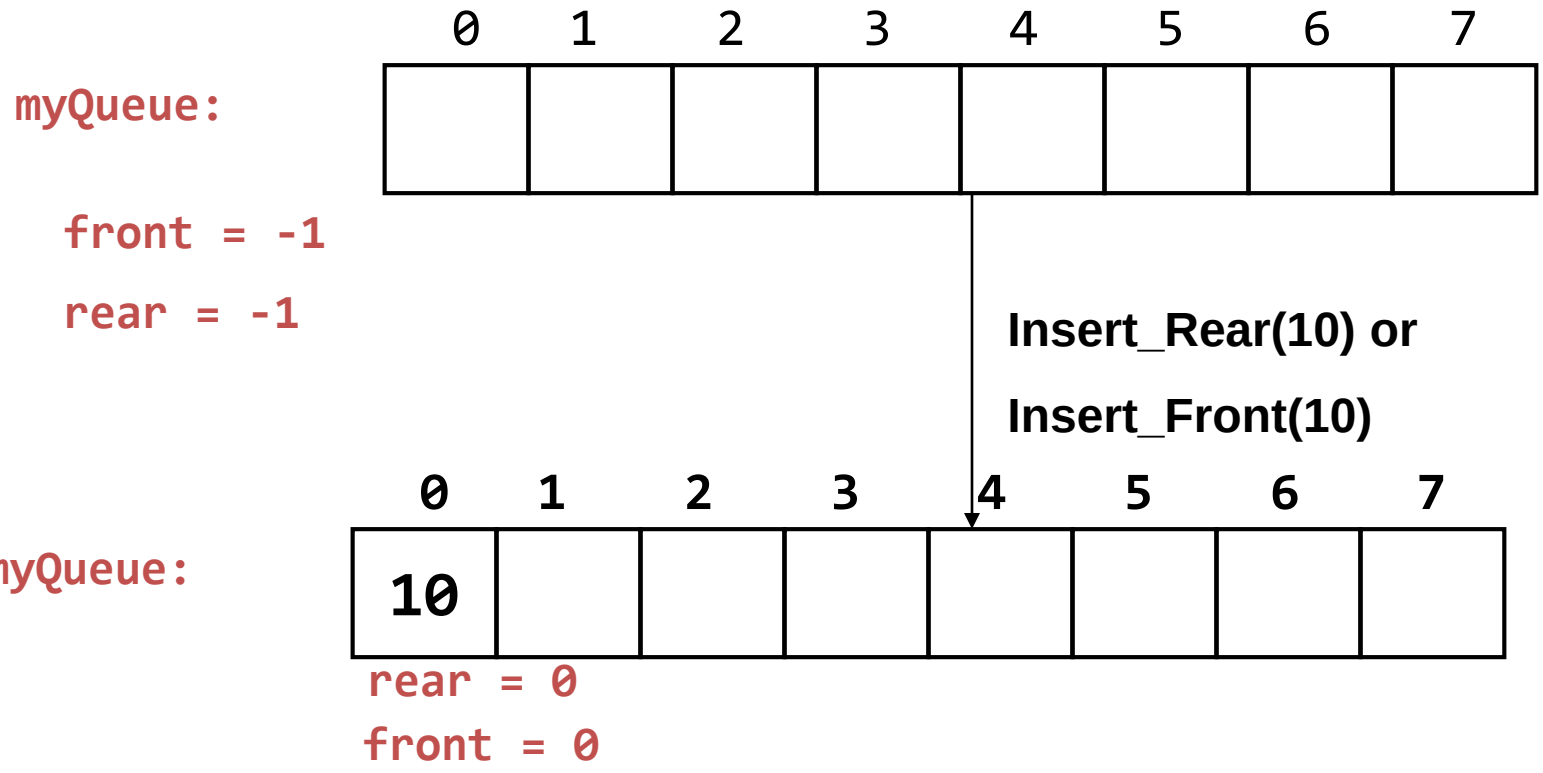
   front = front + 1 .

4. If you insert the element at front end than firstly decrease the front by one i.e

   front = front - 1 than insert the element at front end.

Special Cases in Deque

1.

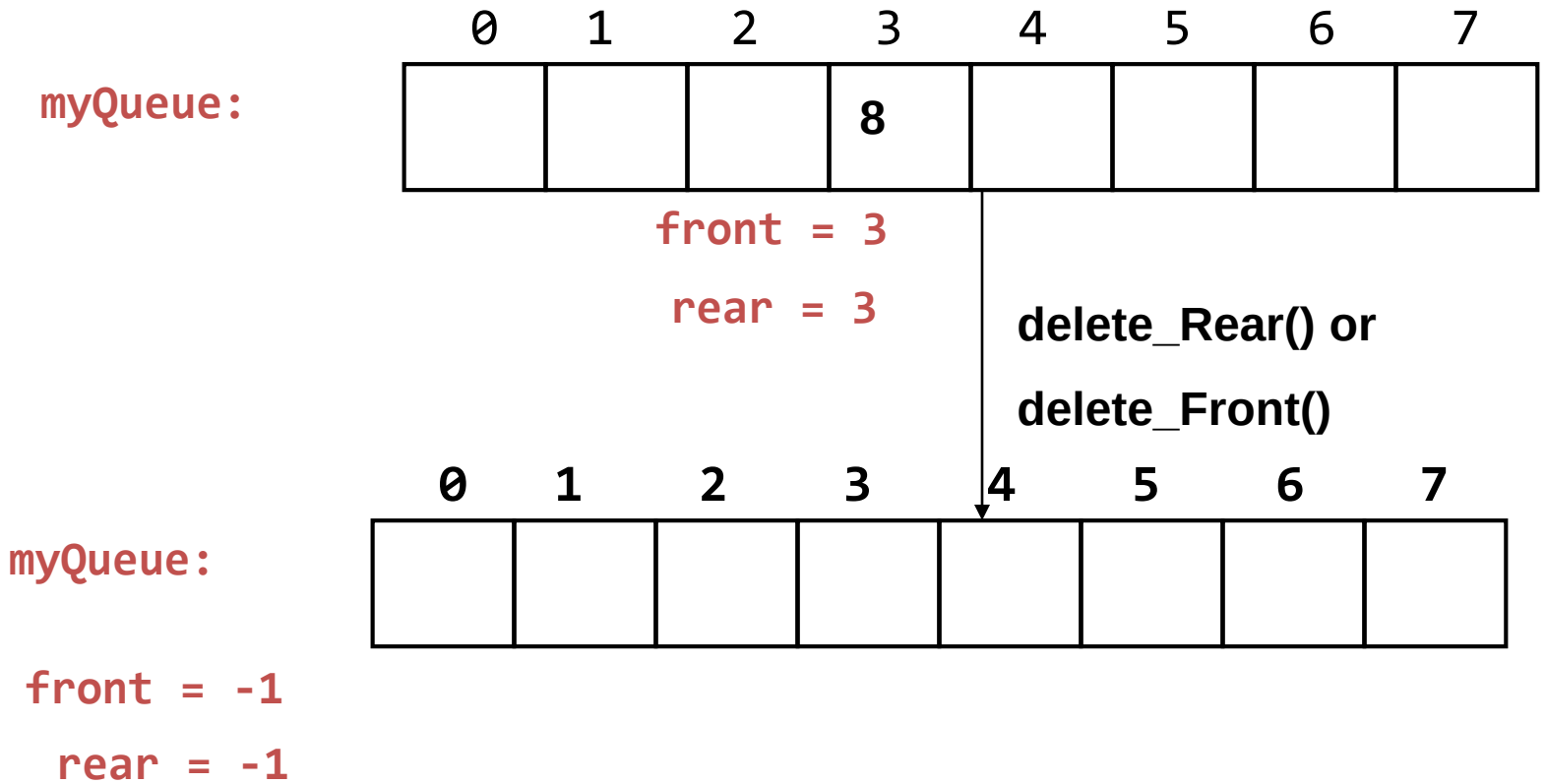


rear = rear +1 = -1+1 = 0

front = front +1 = -1+1 = 0

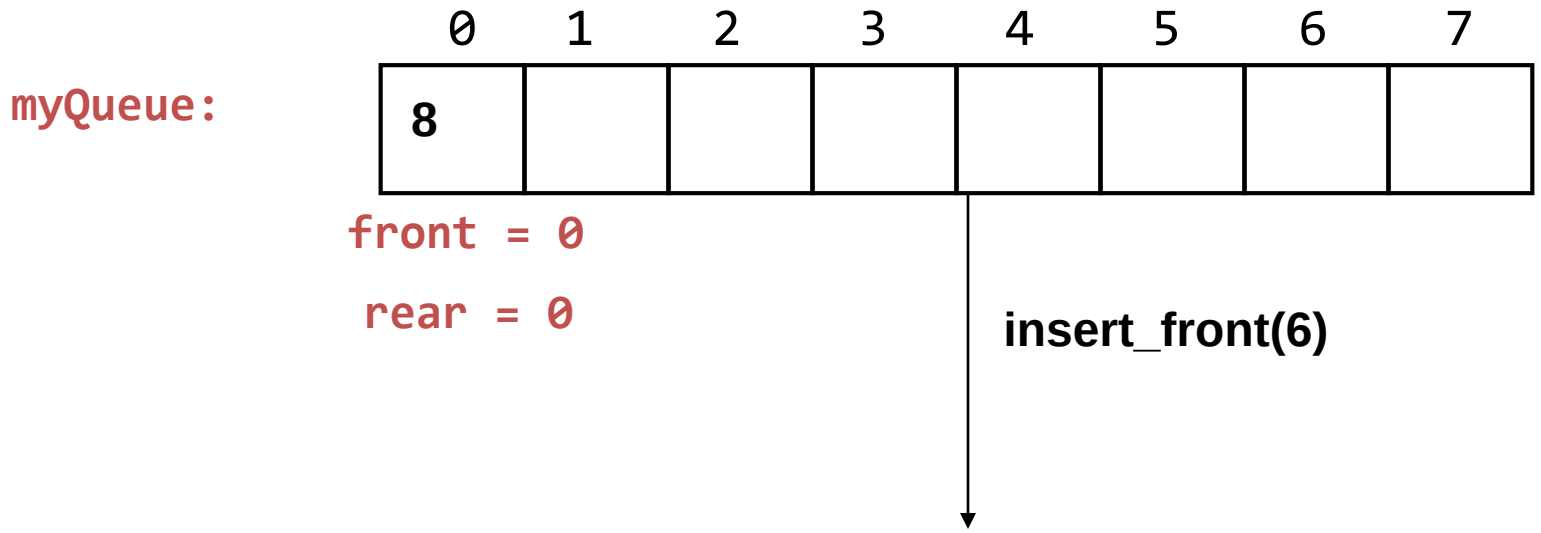
Special Cases in Deque

2.



Special Cases in Deque

3.

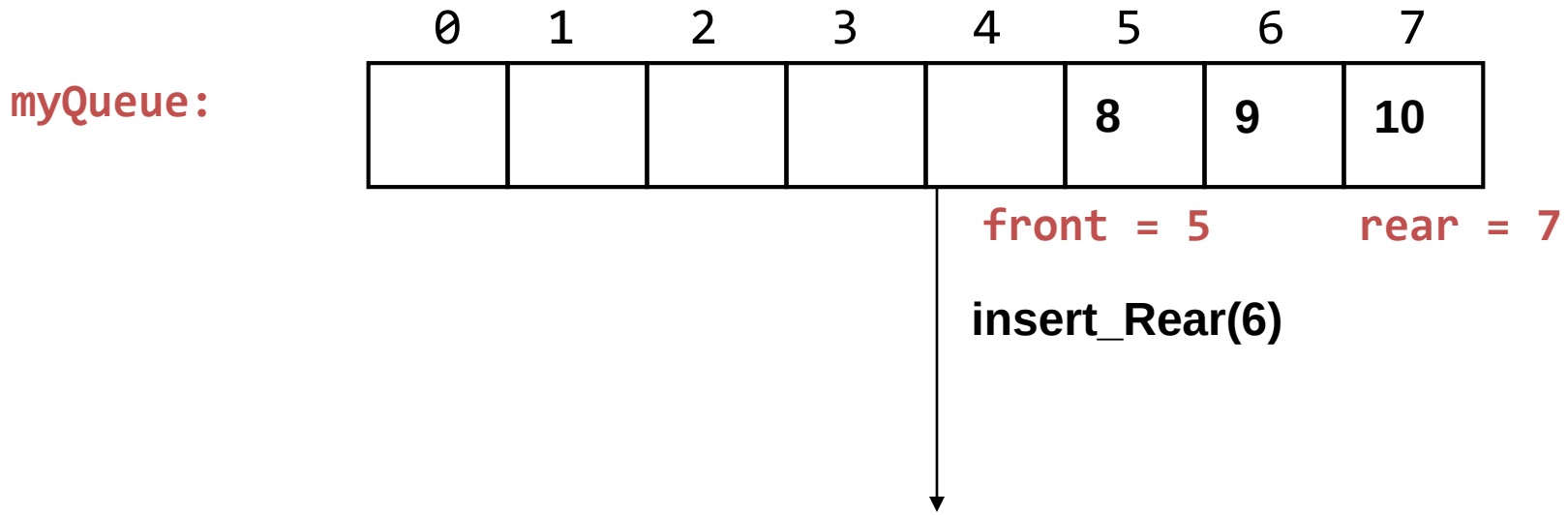


We can not insert at front if we are using simple queue because here deque is overflow and not in circular queue.

In this example we are not using circular queue so we can not insert_front(6) here.

Special Cases in Deque

4.

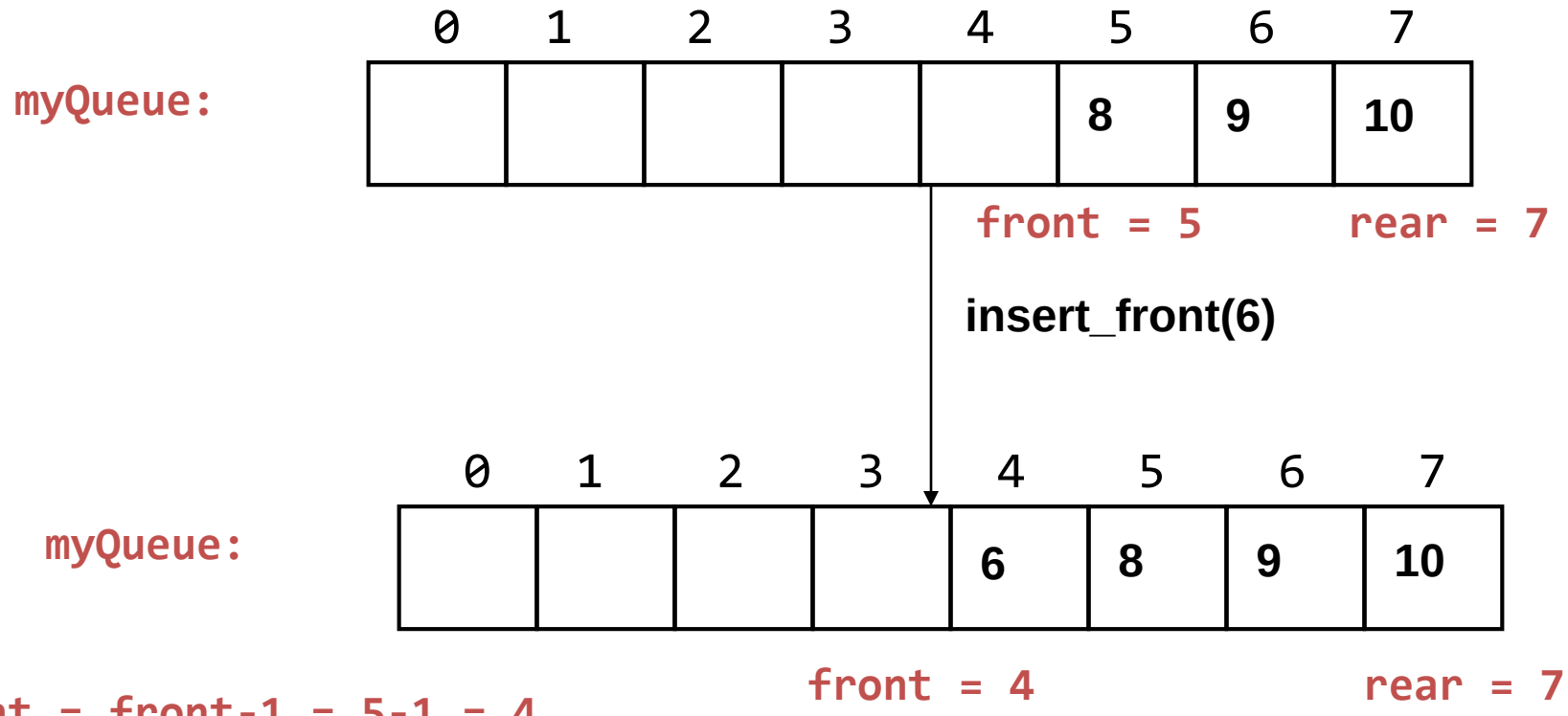


We can not insert at rear end if we are using simple queue because here deque is overflow and not in circular queue.

In this example we are not using circular queue so we can not insert_Rear(6) here.

Cases in Deque

5.

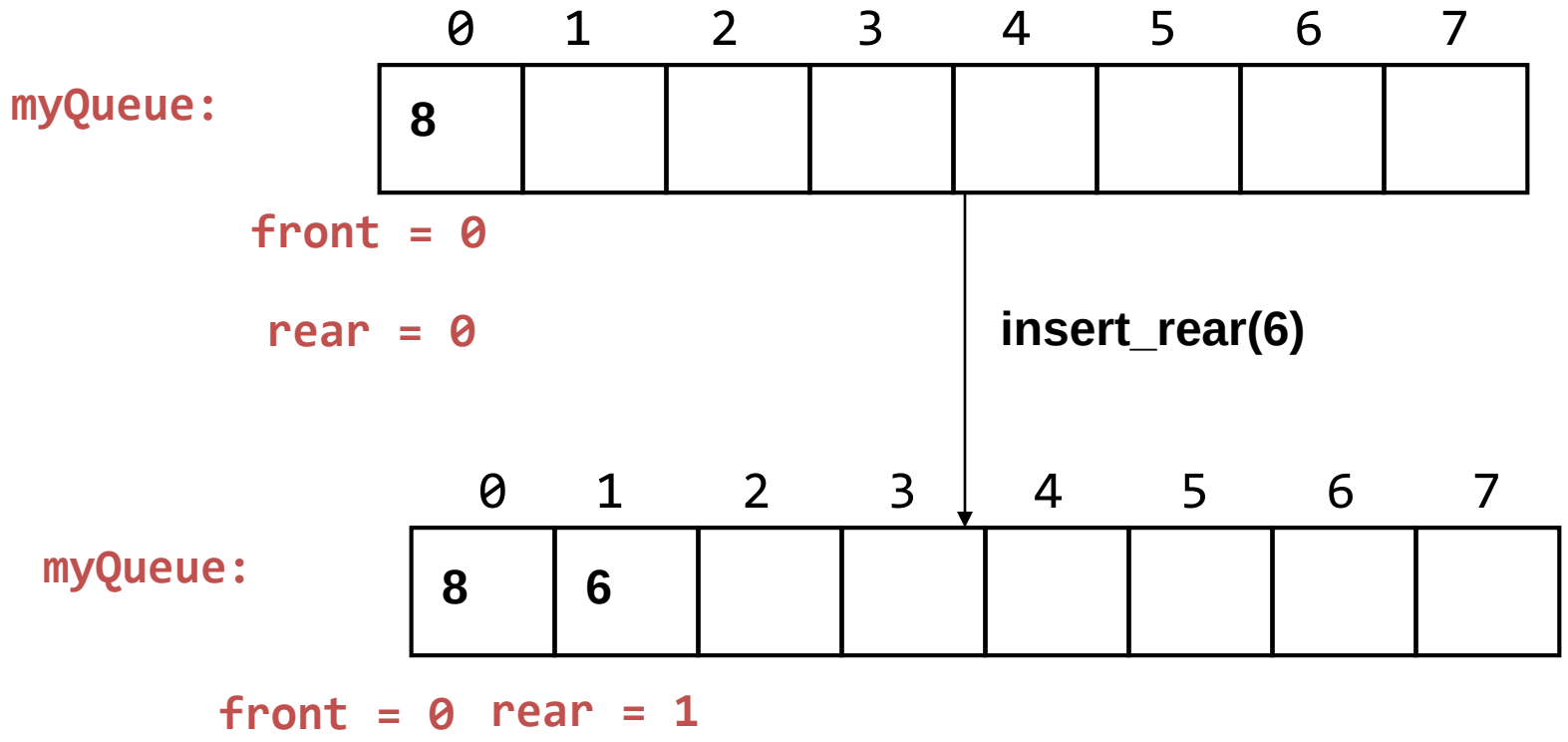


front = front-1 = 5-1 = 4

myQueue[front] = myQueue[4] = 6

Cases in Deque

6.



rear = rear+1 = 0 + 1 = 1

myQueue[rear] = myQueue[1] = 6

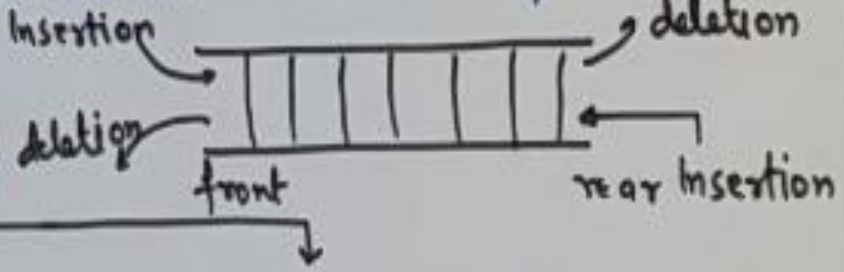
Types of Deque

- Double Ended Queue can be represented in TWO ways.
  - **Input Restricted Double Ended Queue**
  - **Output Restricted Double Ended Queue**

GLA UNIVERSITY MATHURA
Accredited with A+ Grade by NAAC

DEQUE (Double Ended Queue)

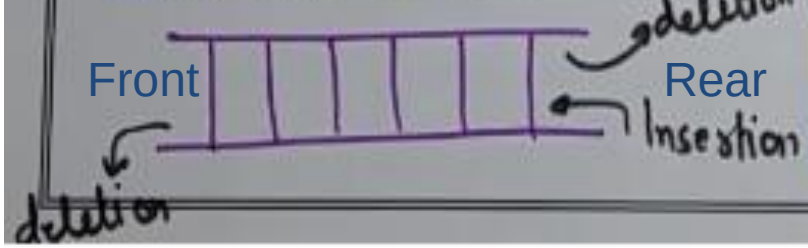
→ In this insertion and deletion can be performed at both end i.e., front pointer can be used for insertion and rear pointer can be used for deletion.



Variations

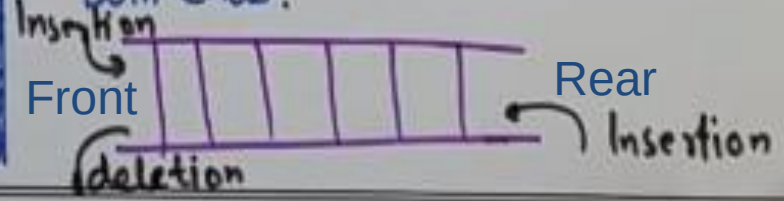
Input Restricted Queue

- deletion can be performed at both ends.
- Insertion at One end (rear)



Output Restricted Queue

- deletion at one end (Front)
- Insertion can be performed at both ends.



Example:

Consider the following deque is allocated 10 rooms

Front=3 Rear=6

DEQUE: ____, ____, _____, England, Australia, Newzeland, Pakistan,_____, _____, _____

Describe the DEQUEUE including front and rear as following operations take place

(a) Bangladesh is added at front
(b) India is added at rear
(c) Sri Lanka is added at rear
(d) Two countries are deleted from front
(e) West indies is added at rear
(f) South Africa is added at rear
(g) Two countries are deleted from rear
(h) Netherland is added at front
(i) Canada is added to the rear
(j) Four countries are deleted from front
(k) Two countries are deleted from rear

Applications of Queue Data Structure

1) When a resource is shared among multiple consumers.

Examples: CPU scheduling, Disk Scheduling.

2) Round-Robin technique for processor scheduling is implemented using queue.

Cont…

3) In a real life scenario, Call Center phone systems uses Queues to hold people calling them in an order, until a service representative is free.